

Memoria del proyecto

Wiki Star Wars - Talleres de Ionic

Gestion de la Calidad del Software (34041)

Grado en Ingenieria Informatica - 4o curso

Universidad de Alicante

Autores

Angel Gonjar Verdejo

Erardo Aldana Pessoa

Fecha de entrega: 19 de mayo de 2026

Indice

Indice.....	2
1. Introduccion	3
2. Metodologia de trabajo	3
2.1. Entorno de desarrollo	3
2.2. Stack tecnico	3
3. Taller 1 - Primera app con Ionic	5
3.1. Objetivos	5
3.2. Tareas realizadas.....	5
3.3. Tiempo total	5
3.4. Problemas encontrados	5
4. Taller 2 - Componentes y servicios	5
4.1. Objetivos	5
4.2. Tareas realizadas.....	6
4.3. Tiempo total	6
4.4. Problemas encontrados	6
5. Taller 3 - Enrutado y formularios	7
5.1. Objetivos	7
5.2. Tareas - Parte 1: enrutado	7
5.3. Tareas - Parte 2: formulario login	7
5.4. Tiempo total	8
5.5. Problemas encontrados	8
6. Taller 4 - Menus y almacenamiento.....	8
6.1. Objetivos	8
6.2. Tareas realizadas.....	8
6.3. Tiempo total	9
6.4. Problemas encontrados	9
7. Mejoras de calidad anadidas (extra).....	9
7.1. Autenticacion completa	9
7.2. UX y robustez ante errores de red.....	9
7.3. Buscador en la lista de articulos	9
7.4. Swipe-to-delete + pull-to-refresh en Favorites.....	10
7.5. Accesibilidad	10
8. Estandares y calidad.....	10
9. Conclusiones	10
10. Referencias.....	11

1. Introduccion

Este documento recoge la experiencia de desarrollo del proyecto realizado durante los cuatro talleres de Ionic de la asignatura Gestion de la Calidad del Software (GCS, 34041). El objetivo es construir, de forma incremental a lo largo de los talleres, una aplicacion movil hibrida tipo wiki que consulta datos del universo Star Wars usando la API publica SWAPI (<https://swapi.tech>).

La aplicacion permite navegar por categorias (People, Planets, Species, Starships), explorar el listado de elementos de cada categoria, consultar fichas individuales, autenticarse con un formulario, marcar articulos como favoritos y consultarlos posteriormente desde una pestana dedicada. Ademas se han anadido mejoras de calidad sobre el guion oficial: AuthGuard con sesion persistente, manejo robusto de errores de red, buscador, swipe-to-delete y atributos de accesibilidad.

La memoria se organiza por talleres siguiendo el orden cronologico de desarrollo. Cada apartado describe las tareas realizadas, el tiempo invertido y los problemas encontrados con sus respectivas soluciones.

2. Metodologia de trabajo

Hemos seguido un enfoque tradicional con consultas puntuales a IA. El PDF de cada taller actuo como guion principal: cada miembro de la pareja iba completando las paginas del documento siguiendo la secuencia indicada, y se consultaba a la IA solo cuando aparecia un bloqueo concreto (errores de compilacion, dudas sobre la API de Ionic 7 frente a versiones anteriores, configuracion de Docker, etc.).

La asistencia de IA se limito a:

- Aclarar diferencias de sintaxis entre Ionic 1/2 y Ionic 7 (por ejemplo float-right vs slot="end").
- Resolver errores concretos de routing y configuracion de modulos lazy-loaded.
- Generar las descripciones de los modelos con los tipos exactos consultados en la documentacion de SWAPI.
- Revisar buenas practicas para la mejora final (AuthGuard, manejo de errores, accesibilidad).

El reparto de trabajo fue por turnos: tras cada taller se hacia una revision conjunta y se aclaraban dudas antes de pasar al siguiente. El control de versiones se llevo en un repositorio Git compartido en GitHub (<https://github.com/agv114-ua/ionic-proyecto-4--final-gcs>) con commits por taller.

2.1. Entorno de desarrollo

Para evitar problemas de versiones de Node, Angular CLI o Ionic CLI entre las dos maquinas se utilizo Docker desde el principio:

- Imagen base: node:21-alpine + ionic/cli@7.
- Volumen montado sobre la carpeta del proyecto.
- docker-compose con un servicio "taller_ionic" que expone el puerto 8100.
- Todos los comandos del CLI (ionic g page, ionic g service, etc.) se ejecutan mediante docker compose run --rm taller_ionic "<comando>".

Esto nos garantiza que los dos integrantes obtuviesen exactamente la misma compilacion y permitio reanudar el trabajo en cualquier maquina sin reinstalar dependencias.

2.2. Stack tecnico

Componente	Version / detalle
------------	-------------------

Node.js	21.x (imagen Docker oficial)
Angular	17.0.2
Ionic	7.5.0
Ionic CLI	7
Storage	@ionic/storage-angular 4.0.0 (añadido en T4)
API externa	SWAPI - https://swapi.tech
Control de versiones	Git + GitHub

3. Taller 1 - Primera app con Ionic

3.1. Objetivos

Crear el esqueleto inicial de la aplicacion con la plantilla "tabs", sustituir las pestanas por defecto (tab1, tab2, tab3) por las propias (wiki, about, exit) y maquetar el contenido estatico de cada una. Incluye tambien la lista de categorias de la wiki como vector hard-coded.

3.2. Tareas realizadas

Tarea	Descripcion	Tiempo	Estado
T1.1	Configurar Docker + docker-compose con node:21-alpine e Ionic CLI 7	20 min	OK
T1.2	Generar el proyecto con plantilla tabs (ionic start applonic tabs)	10 min	OK
T1.3	Eliminar paginas tab1, tab2, tab3 y crear wiki, about, exit	20 min	OK
T1.4	Modificar tabs.page.html y tabs-routing con las nuevas rutas	15 min	OK
T1.5	Pagina Wiki con ion-list de categorias (vector readonly)	15 min	OK
T1.6	Pagina About con ion-card, logo, descripcion y copyright	15 min	OK
T1.7	Pagina Exit con botones YES (alert) y NO (navega a wiki)	15 min	OK
T1.8	Tarea extra: toast de bienvenida al cargar Wiki	5 min	OK
T1.9	Tarea extra: cuarta pestana Favorites (sin contenido)	5 min	OK

3.3. Tiempo total

Aproximadamente 2 horas (entre los dos integrantes).

3.4. Problemas encontrados

- La sintaxis del PDF (float-right) corresponde a Ionic 1/2. En Ionic 7 hubo que sustituirla por slot="end".
- Al crear la pestana Favorites no aparecia en la barra inferior hasta anadir su ruta hija en tabs-routing.
- La imagen "sw.png" debia copiarse manualmente a assets/images/categories porque el zip de Moodle solo incluye los avatares de las cuatro categorias.

4. Taller 2 - Componentes y servicios

4.1. Objetivos

Sustituir el vector hard-coded de categorias por un mockup JSON, encapsular la representacion de una categoria en un componente reutilizable (CategoryComponent) con comunicacion @Input/@Output, y por ultimo conectar la app a la API real de SWAPI mediante un servicio inyectable basado en HttpClient.

4.2. Tareas realizadas

Tarea	Descripcion	Tiempo	Estado
T2.1	Modelo Category (name, image, url)	5 min	OK
T2.2	Mockup assets/data/categories.json	5 min	OK
T2.3	Cargar JSON en wiki.page.ts con fetch + asignacion a categories	15 min	OK
T2.4	Generar CategoryComponent con ionic g component	5 min	OK
T2.5	@Input theCategory + @Input selected + @Output clicked	20 min	OK
T2.6	Boton ojo en cada categoria que emite el evento	10 min	OK
T2.7	Declarar CategoryComponent en WikiPageModule	5 min	OK
T2.8	Sustituir el ngFor de la wiki por app-category con bindings	15 min	OK
T2.9	Servicio WikiService con HttpClient + getAllArticles(category)	20 min	OK
T2.10	Importar HttpClientModule + provider en WikiPageModule	10 min	OK
T2.11	getArticles() en CategoryComponent suscrito al servicio	15 min	OK
T2.12	Mostrar los articulos en ion-list cuando selected es true	10 min	OK

4.3. Tiempo total

Aproximadamente 3 horas (entre los dos integrantes).

4.4. Problemas encontrados

- El PDF llama al servicio con this.theCategory.name pero ese valor llega capitalizado ("People"), mientras que SWAPI sirve los endpoints en minuscula ("/people/"). Resultado: 404 silencioso. Solucion: aplicar .toLowerCase() en el componente antes de invocar el servicio.
- Al integrar app-category dentro de la wiki, la primera version no comunicaba el click. Faltaba declarar el componente en el array declarations del modulo (no basta con importarlo).
- El resultado de fetch().then(res.json()) tenia tipo any, y el lint pedia tipar el array. Se opto por mantener categories: Category[] con el cast implicito.

5. Taller 3 - Enrutado y formularios

5.1. Objetivos

Esta es la parte mas amplia del proyecto. Por un lado se anaden rutas dinamicas con parametros para mostrar la ficha detallada de un articulo, y por otro se crea un formulario de login con validacion reactiva y un servicio de usuarios respaldado por un mockup JSON.

5.2. Tareas - Parte 1: enrutado

Tarea	Descripcion	Tiempo	Estado
T3.1	Crear pagina article con ionic g page	5 min	OK
T3.2	Anadir ruta article/:cat/:id en wiki-routing.module	10 min	OK
T3.3	Modelos People, Planet, Species, Starship con los campos de SWAPI	20 min	OK
T3.4	Funcion generateURL(cat, id) en CategoryComponent	5 min	OK
T3.5	Anadir [href] a cada articulo en category.component.html	10 min	OK
T3.6	Anadir getArticle(category, id) en WikiService	10 min	OK
T3.7	Injectar ActivatedRoute y WikiService en article.page.ts	15 min	OK
T3.8	Leer parametros, llamar al servicio y resolver el modelo con switch	25 min	OK
T3.9	Template article.page.html con ngSwitch + ion-badge por cada campo	30 min	OK
T3.10	ion-back-button con defaultHref="/tabs/wiki"	5 min	OK

5.3. Tareas - Parte 2: formulario login

Tarea	Descripcion	Tiempo	Estado
T3.11	Modelo User + mockup users.json con tres cuentas	10 min	OK
T3.12	Servicio UserService con login(email, password)	15 min	OK
T3.13	Crear pagina login con ionic g page	5 min	OK
T3.14	Importar ReactiveFormsModule en LoginPageModule	5 min	OK
T3.15	FormBuilder con validadores required + email + minLength	15 min	OK
T3.16	Plantilla con ion-input + mensajes de error	25 min	OK

T3.17	Boton Entrar [disabled]="!formLogin.valid"	5 min	OK
T3.18	doLogin que invoca el servicio y navega a tabs o muestra error	15 min	OK
T3.19	ion-alert que se abre cuando error != ""	10 min	OK
T3.20	Modificar app-routing para redirigir a /login al arrancar	10 min	OK

5.4. Tiempo total

Aproximadamente 4 horas (entre los dos integrantes).

5.5. Problemas encontrados

- El switch del article.page.ts esperaba la categoria capitalizada ("People", "Planets") pero el endpoint debia llamarse con la minuscula. Solucion: pasar a .toLowerCase() solo la llamada HTTP y mantener category sin tocar para el switch.
- Al cambiar el redirect inicial de "" a "/login", la ruta /tabs/wiki dejo de funcionar con el error NG04002: Cannot match any routes. La causa era que tabs-routing declaraba path: "tabs" cuando el padre ya lo montaba en "tabs", produciendo /tabs/tabs/wiki. Solucion: cambiar a path: "" dentro del modulo hijo.
- El validator minlength en la plantilla aparecia como hasError("minLength") en una version del PDF y como hasError("minlength") en otra. La forma correcta es la segunda (todo minusculas).
- El ion-alert con [isOpen]="error != \"\" no se cerraba al pulsar OK porque faltaba el handler (didDismiss)="error = \"\" .

6. Taller 4 - Menus y almacenamiento

6.1. Objetivos

Añadir un menu lateral (ion-menu) cargado dinamicamente desde un JSON y persistir los articulos marcados como favoritos en almacenamiento local mediante @ionic/storage-angular. Implementar tambien la pestana Favorites para listarlos y poder volver al detalle.

6.2. Tareas realizadas

Tarea	Descripcion	Tiempo	Estado
T4.1	Mockup assets/data/menu.json (Wiki, Favorites, About)	5 min	OK
T4.2	Cargar menu.json en AppComponent.ngOnInit	10 min	OK
T4.3	Añadir ion-menu con menuId="principal" y router-outlet id="main"	15 min	OK
T4.4	Añadir ion-menu-button en headers de las paginas	15 min	OK
T4.5	Injectar MenuController en login y tabs (ionViewWillEnter)	15 min	OK

T4.6	npm install @ionic/storage-angular + IonicStorageModule.forRoot()	15 min	OK
T4.7	Servicio StorageService con init, set, get	15 min	OK
T4.8	Añadir icono estrella + toggleFavorite en article.page	20 min	OK
T4.9	Toast informativo al añadir/quitar favorito	10 min	OK
T4.10	Página Favoritos: cargar storage y mostrar lista con href	20 min	OK
T4.11	generateURL apuntando a /tabs/wiki/article/:cat/:id	5 min	OK

6.3. Tiempo total

Aproximadamente 3 horas (entre los dos integrantes).

6.4. Problemas encontrados

- El servicio Storage requiere await this.storage.create() antes de poder usar get/set. Si se invoca demasiado pronto las llamadas se quedan colgadas. Solución: envolver la promesa de create y que set/get esperen a esa promesa antes de operar.
- cordova-sqlite-storage que aparece en el PDF no funciona en navegador (es un plugin nativo). @ionic/storage-angular cae automáticamente a localforage/IndexedDB, así que omitimos esa dependencia para evitar warnings.
- Al volver de article a Favoritos la lista no se refrescaba por usar solo ngOnInit. Solución: añadir ionViewWillEnter para recargar el storage cada vez que se entra a la pestaña.

7. Mejoras de calidad añadidas (extra)

Tras completar los talleres se incorporaron varias mejoras alineadas con los criterios de calidad de la asignatura: usabilidad, accesibilidad, robustez y mantenibilidad.

7.1. Autenticación completa

- AuthService que persiste el usuario logueado en Storage bajo la clave "currentUser".
- Auto-login: al arrancar la app se lee el storage y si hay sesión previa se entra directamente a /tabs.
- authGuard funcional (estilo Angular 17) que protege /tabs. Si no hay sesión redirige a /login.
- Opción Logout en el menú lateral que limpia el storage y vuelve a /login.

7.2. UX y robustez ante errores de red

- ion-loading mientras se consulta SWAPI en lista y detalle.
- Toast con color danger si la API responde con error.

7.3. Buscador en la lista de artículos

- ion-searchbar con debounce de 200ms que filtra por nombre.
- Mensaje "No results." cuando el filtro no produce coincidencias.

7.4. Swipe-to-delete + pull-to-refresh en Favorites

- ion-item-sliding con boton danger y icono trash para eliminar favoritos.
- ion-refresher que recarga la lista del Storage con pull-to-refresh.
- Toast informativo al eliminar un favorito.

7.5. Accesibilidad

- aria-label en botones accionables que solo contienen un icono.
- aria-hidden="true" en iconos puramente decorativos.
- alt descriptivo en las imagenes de avatar de cada categoria.

8. Estándares y calidad

Durante el desarrollo se aplicaron criterios alineados con los estándares de calidad software que se trabajan en la asignatura.

Característica de calidad	Como se aborda
Usabilidad	Navegacion por tabs + menu lateral, buscador, swipe-to-delete, mensajes de feedback (toast) y estados vacios claros.
Accesibilidad	aria-labels en iconos accionables, aria-hidden en decorativos, alt en imagenes y contraste del tema dark de Ionic.
Fiabilidad	Manejo explicito de errores HTTP con toast, ion-loading durante esperas y estados de "no results" controlados.
Seguridad	AuthGuard sobre /tabs, sesion solo persistida en cliente, sin exponer credenciales en URL.
Mantenibilidad	Separacion de responsabilidades: servicios (Wiki, User, Storage, Auth), componentes reutilizables, modelos tipados.
Portabilidad	Docker garantiza el mismo entorno entre maquinas y para correccion. App lista para empaquetar con Cordova/Capacitor.

9. Conclusiones

El proyecto ha cumplido al pie de la letra los objetivos planteados en los cuatro PDFs y, además, se ha enriquecido con un paquete de mejoras de calidad que se considera relevante para la asignatura. Trabajar con Ionic 7 + Angular 17 ha permitido cubrir un abanico amplio de conceptos: componentes con comunicacion padre-hijo, servicios inyectables, llamadas HTTP a una API publica, enrutado lazy-loaded con parametros, formularios reactivos con validacion, almacenamiento local persistente y menus laterales.

El uso de Docker desde el inicio ha sido una decision clave: ha eliminado problemas de "en mi maquina funciona" entre los dos integrantes y deja el proyecto listo para correr en cualquier entorno con el mismo comando docker compose up.

Algunas lecciones aprendidas en el camino:

- Las pequenas diferencias entre versiones de Ionic (1/2 vs 7) generan errores poco evidentes; conviene contrastar siempre con la documentacion oficial de la version concreta.
- El enrutado de Angular es estricto: cualquier path repetido o mal compuesto genera errores NG04002 dificiles de depurar a simple vista.

- Pequeñas mejoras (ion-loading, toast de error, swipe-to-delete) elevan mucho la percepción de calidad sin un coste alto de código.
- Tipar correctamente desde el inicio (modelos, parámetros, respuestas) ahorra horas de depuración más adelante.

De cara a futuras iteraciones (talleres siguientes de la asignatura) la app está preparada para integrar UX/UI desde Figma, generar frontend a partir de specs, añadir pruebas E2E con Playwright y empaquetar con Capacitor para distribución móvil.

10. Referencias

- Documentación oficial de Ionic Framework: <https://ionicframework.com/docs>
- Documentación oficial de Angular: <https://angular.io/docs>
- SWAPI - The Star Wars API: <https://www.swapi.tech/documentation>
- @ionic/storage-angular: <https://github.com/ionic-team/ionic-storage>
- Repositorio del proyecto: <https://github.com/agv114-ua/ionic-proyecto-4--final-gcs>
- PDFs oficiales de los talleres 1-4 (Moodle GCS - 34041, curso 2025/2026).